

Documentazione tecnica — Schede "Conto Annuale" e sistema dei filtri

Documento interno. Descrive l'architettura dati/UI delle schede del Conto Annuale costruite sul backend reale (Supabase/PostgREST), il funzionamento dei filtri dinamici e i pattern di codice riutilizzabili, con esempi.

Regola d'oro operativa: l'applicazione **non scrive nulla** sul database remoto. Usa esclusivamente **funzioni RPC di sola lettura** (`fa_ca_*`) e la vista `mv_filtri`. Nessuna vista/tabella/funzione viene creata dal front-end.

1. Panoramica architetturale

Il backend reale **non espone tabelle** per le dashboard: espone **funzioni RPC pre-calcolate** (prefisso `fa_ca_*`) che restituiscono i dati già aggregati, e una vista dizionario `mv_filtri` per popolare i menu dei filtri.



Ogni scheda è composta da **tre file**:

Livello	File	Responsabilità
Service	<code>src/services/ca/<scheda>Service.ts</code>	Definisce i tipi di output, costruisce i parametri (<code>buildParams</code>) e invoca le RPC
Hook	<code>src/hooks/useScheda<Scheda>.ts</code>	Wrappa il service in <code>useQuery</code> (cache, stato loading/error)
Componente	<code>src/components/dashboard/<Scheda>Content.tsx</code>	Legge i filtri globali, gestisce il selettore ente, renderizza KPI/grafici/tabelle

Il "cablaggio" alla navigazione avviene in `src/components/dashboard/OperationalContent.tsx` tramite uno `switch(indicator)`.

2. Il client Supabase "non tipizzato"

Le RPC reali e `mv_filtri` non sono presenti nei tipi generati (database). Per non far fallire il type-check di TypeScript, tutte le chiamate passano da un unico cast isolato:

```
// src/integrations/supabase/untyped.ts
import type { SupabaseClient } from "@supabase/supabase-js";
import { supabase } from "@integrations/supabase/client";

// Unico punto in cui si "spegne" la tipizzazione dello schema.
export const sbUntyped = supabase as unknown as SupabaseClient;
```

Così i service restano puliti e definiscono **a mano** le interfacce TypeScript dei campi restituiti da ogni RPC.

3. Il sistema dei filtri

3.1 La vista dizionario `mv_filtri`

Tutti i menu dei filtri leggono da `public.mv_filtri` (query dirette, non RPC). La vista è **partizionata per anno** e usa un modello a chiave/padre:

Colonna	Significato
tipo	anno, comparto, macrocategoria, categoria, regione, ente, codice_fiscale
anno	anno di riferimento della partizione
chiave	chiave tecnica della voce (es. comparto:FL)
codice	codice usato come valore del filtro (inviato alle RPC)
descrizione	etichetta mostrata all'utente
chiave_padre	collega la voce al genitore nella cascata
ordine	ordinamento

3.2 Service filtri — `services/ca/filtriService.ts`

La cascata è **Comparto** → **Macrocategoria** → **Categoria**; Anno e Regione sono indipendenti. La funzione generica pulisce anche eventuali doppi apici nelle descrizioni della fonte.

```
async function query(tipo, anno, chiavePadre?) {
  let q = sbUntyped.from("mv_filtri")
    .select("chiave, codice, descrizione, chiave_padre")
    .eq("tipo", tipo).eq("anno", anno)
    .order("ordine", { ascending: true })
    .order("descrizione", { ascending: true });
  if (chiavePadre) q = q.eq("chiave_padre", chiavePadre);
  const { data, error } = await q;
  if (error) throw error;
  return (data ?? []).map(/* ... clean(descrizione) ... */);
}

export const fetchComparti      = (anno) => query("comparto", anno);
export const fetchMacrocategorie = (anno, chiaveComparto) => query("macrocategoria", anno, chiaveComparto);
export const fetchCategorie     = (anno, chiaveMacro)    => query("categoria", anno, chiaveMacro);
export const fetchRegioni      = (anno) => query("regione", anno);
```

Il selettore **Ente** (autocomplete, riservato al DFP) usa `searchEnti(anno, term)` con filtro `ilike` sulla descrizione.

3.3 Stato globale — `contexts/FilterContext.tsx`

Uno stato React condiviso da tutta la dashboard. I valori "neutri" sono stringhe ("Tutti"/"Tutte"), l'anno di default è "2023".

```
export interface FilterState {
  macrocategoria: string; categoria: string; comparto: string;
  regione: string; genere: string; anno: string;
  dimensione_pa: string; cluster: string;
}
const defaultFilters = { macrocategoria:"Tutte", categoria:"Tutte",
  comparto:"Tutti", regione:"Tutte", genere:"Tutti", anno:"2023", ... };

// setFilter(key, value) aggiorna un singolo filtro
// activeCount conta quanti filtri divergono dal default (per il badge "Reset")
```

3.4 UI filtri — `components/dashboard/FilterPills.tsx`

Barra unica di "pillole". Punti chiave:

- **Cascata con reset a catena:** cambiando Comparto si azzerano Macrocategoria e Categoria; cambiando Anno si azzerava l'intera cascata (perché `mv_filtri` è partizionata per anno).

```
tsx onChange={v => { setFilter("comparto", v); setFilter("macrocategoria", "Tutte"); setFilter("categoria", "Tutte"); }}
```

- **Query condizionate (enabled):** le macrocategorie si caricano solo se un comparto è selezionato; le categorie solo se esiste la chiave della macro.

```
tsx const macroQ = useQuery({ queryKey: ["mvf", "macro", anno, filters.comparto], queryFn: () => fetchMacrocategorie(anno,
  `comparto:${filters.comparto}`), enabled: filters.comparto !== "Tutti", }); // la categoria dipende dalla CHIAVE della macro
selezionata, non dal codice const macroChiave = (macroQ.data ?? []).find(m => m.codice === filters.macrocategoria)?.chiave;
```

- **Ruolo utente:** gli utenti ente_hr non vedono le pillole Comparto/Regione (!isEnteHr), perché il loro perimetro dati è vincolato all'ente.

4. Il pattern Service (RPC)

Tutti i service `ca/*` condividono lo stesso scheletro.

4.1 buildParams — la regola dei parametri

Regola generale invariata: una chiave **non entra** nell'oggetto se il valore è `null`, `undefined` o stringa vuota. Una `""` arriverebbe a PostgREST come stringa vuota (non come `NULL`) azzerando il risultato.

```
function buildParams(f, opts = {}) {
  const p = {};
  if (f.anno != null)      p.p_anno = f.anno;
  if (f.istituzione)      p.p_istituzione = f.istituzione; // omissso se non ente
  if (f.codiceFiscale)    p.p_codice_fiscale = f.codiceFiscale;
  if (f.comparto)         p.p_comparto = f.comparto;
  if (f.macrocategoria)   p.p_macrocategoria = f.macrocategoria;
  if (f.categoria)        p.p_categoria = f.categoria;
  if (f.regione)          p.p_regione = f.regione;
  // genere: 'T' = tutti => si omette (ramo ELSE lato funzione)
  if (opts.includeGenere !== false && f.genere && f.genere !== "T") p.p_genere = f.genere;
  return p;
}
```

Nota sull'ordine posizionale: alcune firme SQL hanno `p_regione` in quinta posizione (diverso tra schede). Con `supabase-js/REST` i parametri sono **per nome**, quindi l'ordine nel codice **non conta**.

4.2 Helper di invocazione

```
async function rpcRows<T>(fn, params): Promise<T[]> {
  const { data, error } = await sbUntyped.rpc(fn, params);
  if (error) throw error;
  return (data ?? []) as T[];
}
async function rpcOne<T>(fn, params): Promise<T | null> {
  return (await rpcRows<T>(fn, params))[0] ?? null; // KPI a riga singola
}
```

4.3 Esempio completo (scheda Anzianità)

```
export interface AnzianitaKpi {
  personale: number;
  fascia_prevalente: string; // resa così com'è (nessuna mappatura client)
  fascia_prevalente_pct: number; // già in scala %, NON moltiplicare
  anzianita_media: number;
}

export const fetchAnzianitaKpi = (f) =>
  rpcOne<AnzianitaKpi>("fa_ca_anzianita_kpi", buildParams(f));

// p_genere ESCLUSO (ignorato dalla funzione)
export const fetchAnzianitaFasce = (f) =>
  rpcRows<AnzFasciaRow>("fa_ca_anzianita_fasce_genere",
    buildParams(f, { includeGenere: false }));
```

5. Il pattern Hook (React Query)

Hook sottili che incapsulano cache e stato asincrono. La `queryKey` include i filtri, così al cambio filtro React Query rifà la chiamata e aggiorna la UI.

```
const key = (name, f) => ["ca-anzianita", name, f] as const;

export function useAnzianitaKpi(f) { return useQuery({ queryKey: key("kpi", f), queryFn: () => fetchAnzianitaKpi(f) }); }
export function useAnzianitaFasce(f) { return useQuery({ queryKey: key("fasce", f), queryFn: () => fetchAnzianitaFasce(f) }); }
export function useAnzianitaEvoluzione(f){ return useQuery({ queryKey: key("evoluzione", f), queryFn: () => fetchAnzianitaEvoluzione(f) }); }
```

6. Il pattern Componente (`...Content.tsx`)

Struttura ricorrente:

1. Legge i filtri globali e il profilo utente.
2. Mappa i valori "neutri" della UI ("Tutti"/"Tutte") su null.
3. Espone il **selettore Ente** solo al DFP.
4. Invoca gli hook e renderizza KPI + grafici + tabella.

```
const { filters } = useFilters();
const { profile } = useAuth();
const isDfp = profile?.role === "dfp";

const gMap = { Tutti: "T", Uomini: "U", Donne: "D" };
const filtri = {
  anno: Number(filters.anno) || 2023,
  istituzione: ente?.codice ?? null, // dal selettore Ente (DFP)
  comparto: filters.comparto !== "Tutti" ? filters.comparto : null,
  macrocategoria: filters.macrocategoria !== "Tutte" ? filters.macrocategoria : null,
  categoria: filters.categoria !== "Tutte" ? filters.categoria : null,
  regione: filters.regione !== "Tutte" ? filters.regione : null,
  genere: gMap[filters.genere] ?? "T",
};

const kpi = useAnzianitaKpi(filtri); // → 4 card
const fasce = useAnzianitaFasce(filtri); // → bar chart + tabella
```

Il **selettore Ente** (DFP) è un autocomplete che pilota `p_istituzione`:

```
const entiQ = useQuery({
  queryKey: ["enti-search", anno, enteTerm],
  queryFn: () => searchEnti(anno, enteTerm),
  enabled: enteTerm.trim().length >= 2,
});
// visibile solo se isDfp; alla selezione: setEnte({ codice, descrizione })
```

Le librerie grafiche sono **Recharts** (BarChart, AreaChart, LineChart, ComposedChart, PieChart).

7. Le schede realizzate

Tutte cablate in `OperationalContent.tsx` (source = "conto-annuale"):

Indicatore (URL)	Componente	RPC usate
analisi-eta	AnalisiEtaContent	fa_ca_personale_servizio, fa_ca_eta, fa_ca_anzianita_media, fa_ca_eta_fasce_genere, fa_ca_eta_evoluzione, fa_ca_eta_benchmark
analisi-anzianita	AnzianitaContent	fa_ca_anzianita_kpi, fa_ca_anzianita_fasce_genere, fa_ca_anzianita_evoluzione
assunti-causale	AssuntiCausaleContent	fa_ca_assunti_kpi, fa_ca_assunti_causali, fa_ca_assunti_evoluzione
cessazioni	CessazioniContent	fa_ca_cessazioni_kpi, fa_ca_cessazioni_causali (x2), fa_ca_cessazioni_evoluzione
tasso-turnover	TassoTurnoverContent	fa_ca_turnover_kpi, fa_ca_turnover_evoluzione
tasso-sostituzione	TassoSostituzioneContent	fa_ca_sostituzione_kpi, fa_ca_sostituzione_evoluzione

7.1 Anzianità di servizio

- **KPI** (1 chiamata, riga singola): Personale totale, Fascia prevalente (testo così com'è), % fascia prevalente (già in scala), Anzianità media.
- **Distribuzione** (bar chart, 5 fasce sempre presenti): sbiadimento del genere non selezionato.
- **Evoluzione** (area chart 100%): formato **lungo** → **pivot lato client** (x=anno, serie=fascia, y=percentuale).
- `p_genere` escluso da `fasce_genere`.

7.2 Assunti per causale

- **KPI:** Totale assunti, Causale prevalente (COALESCE(desc, causale), resa così com'è), % donne (già in scala), Personale (da mv_occupazione).
- **Barre orizzontali** Uomini/Donne + **Donut** (Composizione) da **una sola** risposta.
- **Trend** (area chart, formato largo anno, assunti, nessun pivot).
- Nota p_genere **non uniforme**: agisce su assunti/valore/percentuale/personale, **non** su uomini/donne/tutti. Lo passiamo sempre: le barre restano invariate, donut/KPI riflettono il filtro.

7.3 Cessazioni dal servizio

- **KPI:** Cessati, Assunti, **Saldo** (può essere negativo → segno e colore), Tasso turnover (già in scala).
- **fa_ca_cessazioni_causali** chiamata **DUE volte**, cambia solo p_movimento:
- 'C' → grafico "Cessazioni per causale"
- 'A' → grafico "Assunzioni per causale"
- **Eccezione parametri:** p_movimento va aggiunto **sempre** (fisso per componente), **dopo** i filtri; ammette solo 'A'/'C' maiuscoli.
- p_genere **ignorato** dalle causali → non passato.
- Pulizia dei doppi apici nelle descrizioni (es. "COLLOCAMENTO A RIPOSO...").

```
ts function buildParams(f, opts = {}) { /* ... filtri ... */ if (opts.includeGenere !== false && f.genere && f.genere !== "T")
p.p_genere = f.genere; if (opts.movimento) p.p_movimento = opts.movimento; // ECCEZIONE: sempre presente return p; } export const
fetchCessazioniCausali = (f, movimento) => rpcRows("fa_ca_cessazioni_causali", buildParams(f, { includeGenere: false, movimento
}));
```

7.4 Tasso di turnover

- **KPI** (1 chiamata): Tasso turnover + **delta pp** (può essere NULL), Cessati, Assunti, Saldo (spesso negativo).
- **Combo chart a doppio asse:** barre Assunti/Cessati (asse sx, volumi) + linea Tasso turnover % (asse dx). **ComposedChart** con due **YAxis** (yAxisId="left"/"right").
- **Bar chart Saldo cumulato** (saldo_cumulato, con ReferenceLine y={0}); il cumulato parte dal primo anno presente nella MV.
- **Tabella** serie storica: anno, assunti, cessati, saldo, turnover %, ingresso %.
- p_genere applicato **uniformemente** (nessuna eccezione).

7.5 Tasso di sostituzione

- Contratto: sostituzione = assunti / cessati × 100 (>100 ⇒ ingressi > uscite).
- **KPI** (1 chiamata): Tasso sostituzione (senza sottotitolo), Media periodo (media aritmetica dei tassi annui), **Anni con ricambio positivo** = anni_ricambio_positivo / anni_periodo (composto lato client), Variazione vs anno prec. (pp, può essere NULL).
- **Bar chart** Tasso di sostituzione (serie singola) — **didascalia rimossa** come da nota.
- **Area chart** Assunti vs Cessati.
- **Tabella** con **esattamente 5 colonne**: anno, assunti, cessati, saldo, tasso sostituzione.
- p_anno è sempre il **limite superiore** (m.anno <= p_anno); nel KPI determina anche l'ampiezza del periodo.

8. Note trasversali (postille dei file di mappatura)

Tema	Regola applicata
Parametri vuoti	Mai inviati (buildParams); "" ≠ NULL in PostgREST
p_istituzione / p_codice_fiscale	Omessi se l'accesso non è di un ente (mai '')
Percentuali	Molte RPC restituiscono valori già in scala % : non moltiplicare per 100
Etichette testuali	fascia_prevalente, causale_prevalente → renderizzate così come arrivano
p_genere non uniforme	Assunti: agisce su valore/percentuale, non su uomini/donne. Cessazioni causali: ignorato. Turnover/Sostituzione: uniforme
p_movimento	Solo Cessazioni causali; sempre presente; solo 'A'/'C'
Saldo negativo	KPI e assi prevedono valori < 0
Formato lungo vs largo	Anzianità evoluzione = lungo (pivot client); Assunti/Cessazioni/Turnover/Sostituzione = largo (no pivot)
Descrizioni con apici	Ripulite con clean()

9. Come aggiungere una nuova scheda (checklist)

1. **Leggere l'Excel di mappatura:** RPC, parametri, campi di output e **tutte le note** (colonne "Note"/"punti aperti").

2. **Verificare le RPC** contro il DB live in sola lettura (`/rest/v1/rpc/...`).
3. Creare `services/ca/<scheda>Service.ts`: interfacce output + `buildParams` + `fetcher` (`rpcOne` per i KPI a riga singola, `rpcRows` per liste/serie).
4. Creare `hooks/useScheda<Scheda>.ts` con gli `useQuery`.
5. Creare `components/dashboard/<Scheda>Content.tsx` (KPI/grafici/tabelle + selettore Ente per il DFP).
6. Cablare in `operationalContent.tsx` (`switch(indicator)`), e se serve aggiungere/rimuovere la voce in `AppSidebar.tsx`.
7. `npx tsc --noEmit` e verifica UI.

10. Ambiente e sicurezza

- Le variabili reali (`VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`, `VITE_KEYCLOAK_*`) stanno in `.env` (gitignored). Riferimenti in `docs/ENV_LOCALE.md`. **Non committare mai le credenziali.**
- Autenticazione: **Keycloak** quando le `VITE_KEYCLOAK_*` sono valorizzate, altrimenti login dimostrativo (mock) via `sessionStorage`.
- Multi-tenancy (in roadmap): per gli utenti `ente_hr` si inietterà `p_codice_fiscale` leggendo il claim `cf_ente` dal token Keycloak.
- **Nessuna operazione di scrittura** viene mai effettuata sul DB remoto: solo chiamate RPC di lettura e `SELECT` su `mv_filtri`.

11. Pulizia del codice morto (eseguita)

Con la migrazione alle RPC sono stati rimossi i componenti mock e le loro dipendenze ormai orfane:

- Sezioni mock: `AnzianitaSection`, `AssuntiCausaleSection`, `CessazioniSection`, `TassoTurnoverSection`, `TassoSostituzioneSection`.
- Hook dati mock: `useAssuntiData`, `useCessatiData` (rimossi anche dal barrel `hooks/index.ts`).
- Service `dw/assuntiService` (rimosso anche da `services/dw/index.ts`).

Mantenuti: `dw/cessatiService` (coperto da test), gli altri service `dw/*` e le sezioni ancora attive non ancora migrate.